



North South University

Department of Electrical and Computer Engineering

CSE 215L (Programming Language II Lab)

Lab 2: Mathematical, String Functions and Conditional Statements

Objective

- **Operators:** Understand and apply arithmetic, assignment, relational, logical, increment/decrement, and conditional operators in Java.
- **Mathematical Functions:** Learn to use common mathematical functions provided by Java's `Math` class, including powers, square roots, maximum/minimum, and rounding functions.
- **String Functions:** Understand and apply common String operations such as length calculation, character access, concatenation, case conversion, and trimming.
- **Conditional Statements:** Understand how programs make decisions using `if`, `if-else`, and multiple conditions.
- **Nested Conditions:** Apply conditional statements inside other conditional statements to solve multi-level decision-making problems.
- **Switch Case:** Use the `switch` statement to select one operation from multiple possible choices.
- **Problem Solving:** Apply mathematical functions, String operations, and conditional statements to solve basic programming problems.

Operators

Operators are symbols used to perform operations on values and variables. Common Java operators are shown below.

Operator	Purpose	Example	Result
<code>+, -, *, /, %</code>	Perform basic arithmetic operations.	<code>10 + 3, 10 % 3</code>	<code>13, 1</code>
<code>=, +=, -=, *=, /=</code>	Assign or update the value of a variable.	<code>x = 10; x += 5;</code>	<code>15</code>
<code>==, !=, >, <, >=, <=</code>	Compare two values.	<code>10 > 5</code>	<code>true</code>
<code>&&, , !</code>	Combine or reverse boolean conditions.	<code>(x > 5) && (x < 10)</code>	<code>true/false</code>
<code>++, --</code>	Increase or decrease a value by one.	<code>x++; x--;</code>	<code>x+1, x-1</code>
<code>?:</code>	Select one of two values based on a condition.	<code>(x > 0) ? 1 : -1</code>	<code>1 or -1</code>

Operator Precedence

When an expression contains multiple operators, Java follows operator precedence rules. Parentheses can be used to make the intended order clear.

```
int result = 10 + 5 * 2;    // 20
int result2 = (10 + 5) * 2; // 30
```

Common Mathematical Functions

Java provides many mathematical functions through the `Math` class. These functions are useful for calculations involving powers, square roots, rounding, trigonometry, and finding maximum or minimum values.

Two commonly used constants are:

```
Math.PI
Math.E
```

`Math.PI` represents π , while `Math.E` represents the base of the natural logarithm.

Some commonly used functions are:

Function	Purpose	Example	Output
<code>Math.sqrt(x)</code>	Returns the square root of x .	<code>Math.sqrt(25)</code>	5.0
<code>Math.pow(x,y)</code>	Returns x raised to the power y .	<code>Math.pow(2,3)</code>	8.0
<code>Math.abs(x)</code>	Returns the absolute value of x .	<code>Math.abs(-10)</code>	10
<code>Math.max(x,y)</code>	Returns the larger of two values.	<code>Math.max(10,20)</code>	20
<code>Math.min(x,y)</code>	Returns the smaller of two values.	<code>Math.min(10,20)</code>	10
<code>Math.ceil(x)</code>	Rounds upward to the nearest integer value.	<code>Math.ceil(12.3)</code>	13.0
<code>Math.floor(x)</code>	Rounds downward to the nearest integer value.	<code>Math.floor(12.8)</code>	12.0
<code>Math.round(x)</code>	Rounds to the nearest integer.	<code>Math.round(12.6)</code>	13
<code>Math.sin(x)</code>	Returns the sine of an angle in radians.	<code>Math.sin(Math.PI/2)</code>	1.0
<code>Math.cos(x)</code>	Returns the cosine of an angle in radians.	<code>Math.cos(Math.PI)</code>	-1.0
<code>Math.tan(x)</code>	Returns the tangent of an angle in radians.	<code>Math.tan(0)</code>	0.0
<code>Math.toRadians(x)</code>	Converts degrees to radians.	<code>Math.toRadians(180)</code>	3.1416
<code>Math.toDegrees(x)</code>	Converts radians to degrees.	<code>Math.toDegrees(Math.PI)</code>	180.0

String Functions

Strings are used to store text. Java provides several methods through the `String` class for examining and manipulating text.

Common String methods include:

Method	Purpose	Example	Output
<code>length()</code>	Returns the number of characters in a String.	<code>"Java".length()</code>	4
<code>charAt(index)</code>	Returns the character at the specified index.	<code>"Java".charAt(0)</code>	J
<code>concat(str)</code>	Joins another String to the current String.	<code>"Hello ".concat("Java")</code>	Hello Java
<code>toUpperCase()</code>	Converts the String to uppercase.	<code>"Java".toUpperCase()</code>	JAVA
<code>toLowerCase()</code>	Converts the String to lowercase.	<code>"JAVA".toLowerCase()</code>	java
<code>trim()</code>	Removes leading and trailing whitespace.	<code>" Java ".trim()</code>	Java
<code>substring(start,end)</code>	Extracts part of a String.	<code>"Programming".substring(0,7)</code>	Program
<code>equals(str)</code>	Compares two Strings for equality.	<code>"Java".equals("Java")</code>	true

Warning – String Comparison

Do not use `==` to compare String contents. The `==` operator checks whether two String variables refer to the same object, whereas `equals()` compares the actual contents of the Strings.

Rule: Always use `equals()` when comparing String contents.

```
String a = "Java";
String b = new String("Java");

System.out.println(a == b);    // false
System.out.println(a.equals(b)); // true
```

Conditional Statements

A conditional statement allows a program to make decisions based on whether a condition is true or false.

The basic form is:

```
if (condition) {
    // statements
}
```

For example:

```
int age = 20;

if (age >= 18) {
```

```
    System.out.println("Adult");  
}
```

The statements inside the `if` block execute only when the condition is `true`.

if-else

The `if-else` statement provides two possible paths.

```
if (condition) {  
    // true  
} else {  
    // false  
}
```

Example:

```
int number = 7;  
  
if (number % 2 == 0) {  
    System.out.println("Even");  
} else {  
    System.out.println("Odd");  
}
```

Multiple Conditions

When several conditions need to be checked, `else-if` can be used.

```
if (marks >= 80) {  
    System.out.println("A");  
} else if (marks >= 70) {  
    System.out.println("B");  
} else if (marks >= 60) {  
    System.out.println("C");  
} else {  
    System.out.println("F");  
}
```

Conditions are checked from top to bottom. Once a condition is satisfied, the remaining conditions are skipped.

Nested if

An `if` statement placed inside another `if` statement is called a nested conditional.

Example:

```
if (age >= 18) {  
  
    if (hasID) {  
        System.out.println("Access granted");  
    } else {  
        System.out.println("ID required");  
    }  
  
} else {  
    System.out.println("Access denied");  
}
```

Nested conditions are useful when a second decision depends on the result of a previous decision.

Switch Case

The `switch` statement is useful when one expression needs to be compared with several fixed values.

```
int day = 2;  
  
switch (day) {  
  
    case 1:  
        System.out.println("Sunday");  

```

```
        break;

    case 2:
        System.out.println("Monday");
        break;

    case 3:
        System.out.println("Tuesday");
        break;

    default:
        System.out.println("Invalid day");
}
```

The **break** statement prevents execution from continuing into the next case.

The **default** block executes when none of the cases match.

Classwork

Complete the following tasks.

1. Declare four numeric variables and take user input for all four. Find and print their sum.
2. Declare three **float** variables and take user input. Find and print their product.
3. Find the difference between the sum from Task 1 and the product from Task 2.
4. Find the average of the sum and product from the previous tasks, then print the square of the average using **Math.pow()**.
5. Take three scores as user input. Find the maximum and minimum scores using **Math.max()** and **Math.min()**, then calculate their average.
6. Take the salaries for three months as input. Find the maximum salary, round it using an appropriate **Math** function, and add half of the minimum salary to the rounded maximum.
7. Take an integer as input and determine whether it is positive, negative, or zero using **if-else**.
8. Take a student's marks as input and display the grade using **if-else-if**.
9. Take a number from 1–7 and display the corresponding day of the week using **switch**.

Homework

Complete the following tasks.

1. Declare a variable called **name** and assign your name to it.
2. Declare three variables to store exam scores and use user input to assign their values.
3. Calculate the average of the three scores.
4. Print the name and average score in the **SAME LINE**.
5. Take a number from the user and calculate the number raised to the power of $\frac{1}{2}$ using **Math.pow()**.
6. Take a String as input and print:
 - its length,
 - its first character,
 - its uppercase form, and
 - its lowercase form.
7. Take an integer as input and determine whether it is even or odd using **if-else**.
8. Take three numbers as input and find the largest number using nested **if** statements.
9. Create a simple calculator using **switch**. The user should enter two numbers and an operator (+, -, *, or /), and the program should display the result.